

University of Nottingham
UK | CHINA | MALAYSIA

Pointers and Input-Output

Dr. Wooi Ping Cheah

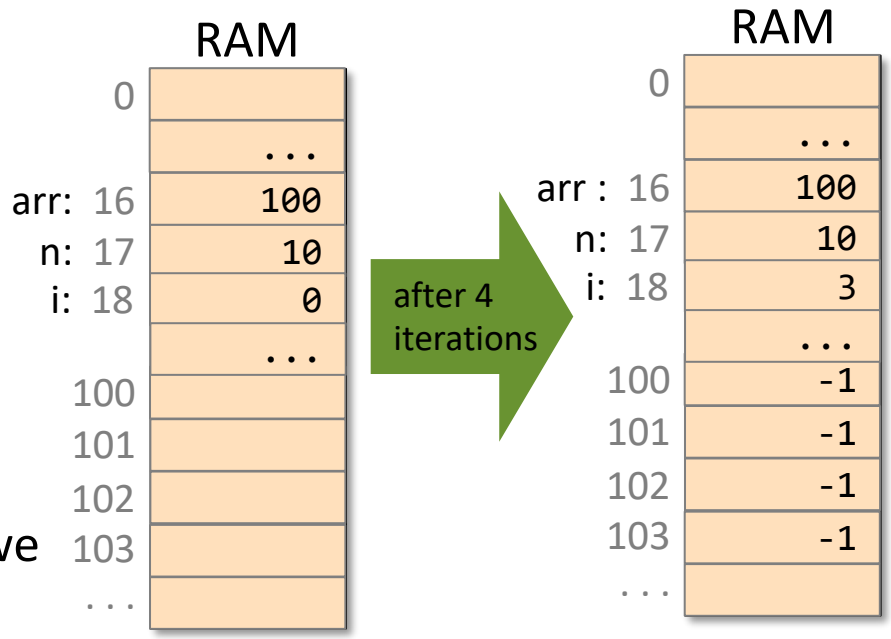
Outlines

- Hack pointers
- Hack input / output

Pointers

Example:

```
// for (i=0; i<n; i++) {
//     arr[i] = -1
// }
```



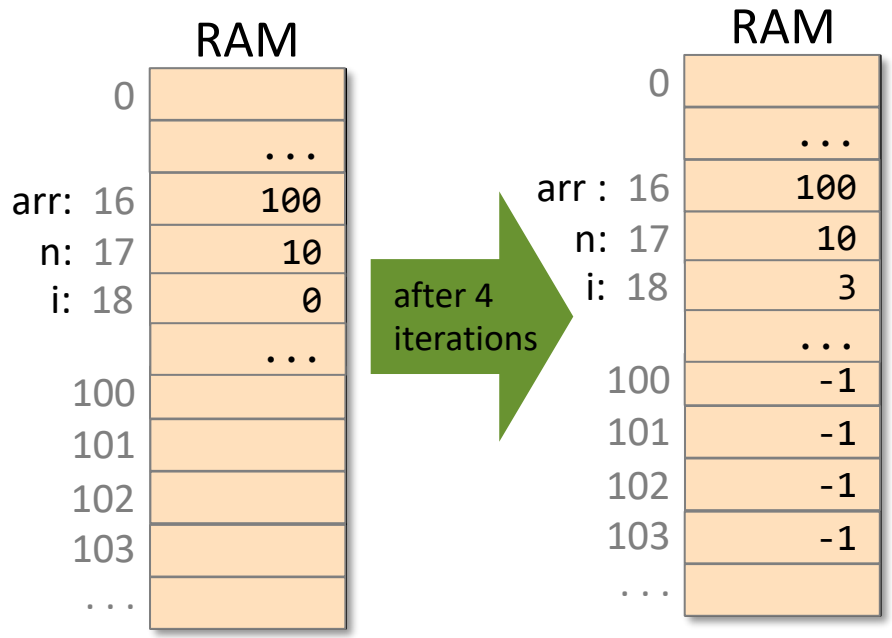
Observations:

- The array is implemented as a **block of memories**.
- To access these memories one by one, we need a variable to hold the current address.
- *Variables that represent addresses* are called **pointers**.
- There is nothing special about pointer variables, except that their values are interpreted as addresses.

Pointers

Example:

```
// for (i=0; i<n; i++) {
//     arr[i] = -1
// }
  // Suppose that arr=100 and n=10
  // Let arr = 100
  @100
  D=A //D = 100
  @arr
  M=D // arr = 100
  // Let n = 10
  @10
  D=A // D = 10
  @n
  M=D // n = 10
  // Let i = 0
  @i
  M=0 // i = 0
  // Loop code continues
  // in next slide...
```

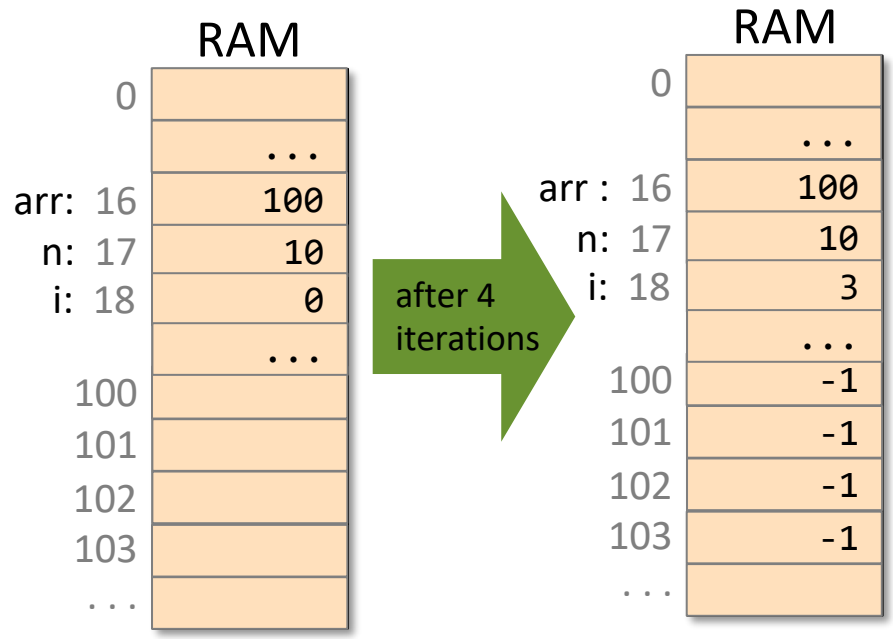


Pointers

Example:

```
(LOOP)
  // if (i==n) goto END
  @i
  D=M // D = i
  @n
  D=D-M // D = i-n
  @END
  D;JEQ // if (i==n) goto END
  // RAM[arr+i] = -1
  @arr
  D=M // D = arr
  @i
  A=D+M // A = arr + i
  M=-1 // M[arr+i] = -1
  // i++
  @i
  M=M+1 // i = i + 1
  // goto LOOP
  @LOOP
  0;JMP
(END)
  @END
  0;JMP // END
```

typical pointer manipulation

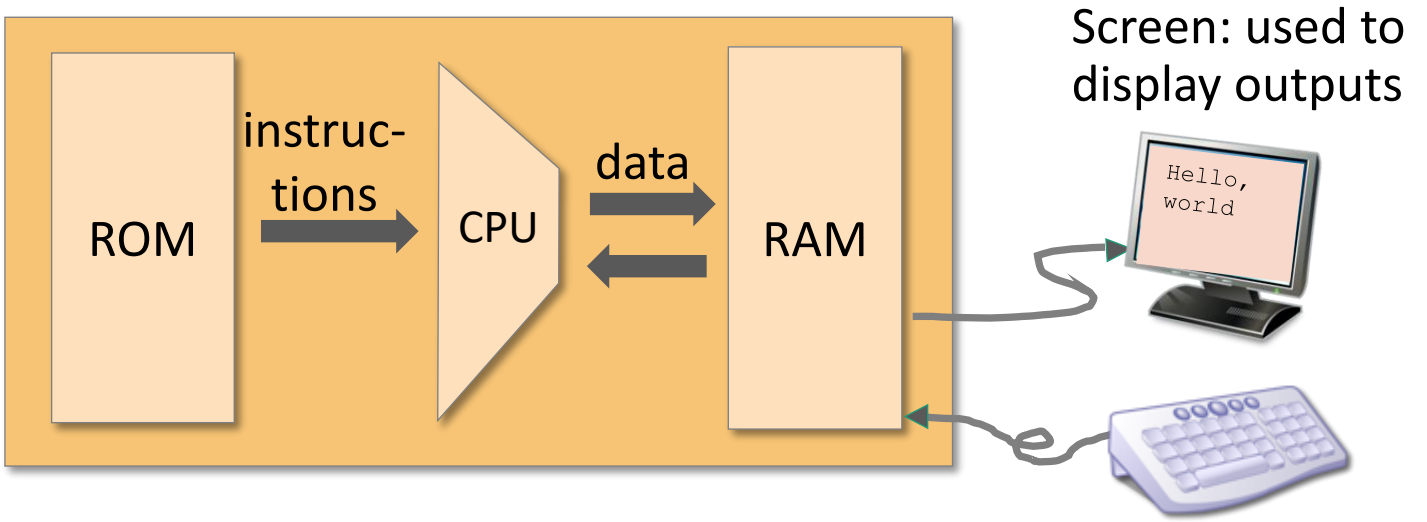


- Pointers: Variables that store memory addresses (like `arr`).
- Pointers in Hack: Whenever we have to access memory using a pointer, we need an instruction like **A = *expression***.
- Semantics:
  "set the address register to some value".

Outlines

- Hack pointers
- Hack input / output

Input / output



Screen: used to display outputs

Keyboard: used to enter inputs

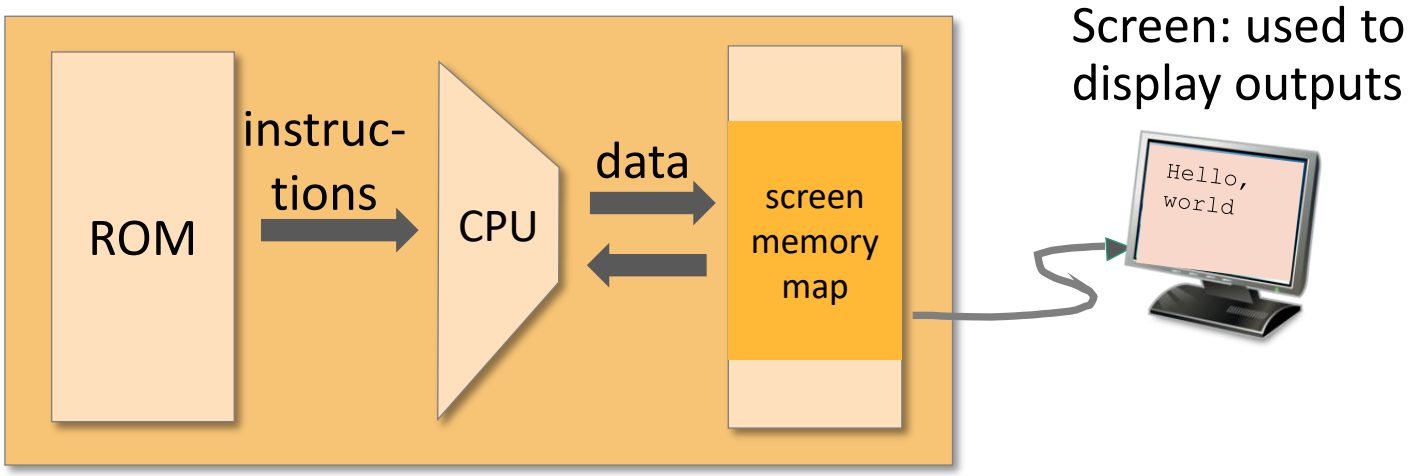
I/O handling (high-level):

Software libraries enabling text, graphics, audio, video, etc.

I/O handling (low-level):

Bit manipulation.

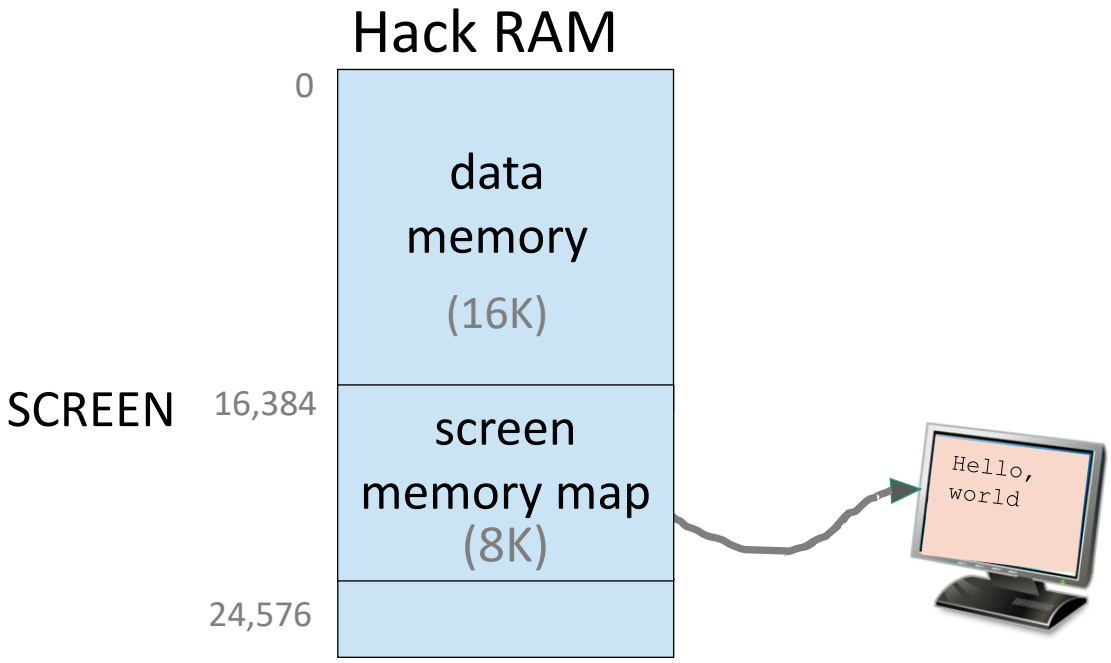
Memory mapped output



Screen: used to display outputs

- Memory mapped output
  - A **designated memory** area to manage a display unit.
  - The physical display is continuously *refreshed* from the memory map, many times per second. (It is slow in Hack computer.)
  - Output is effected by writing code that manipulates the screen memory map.

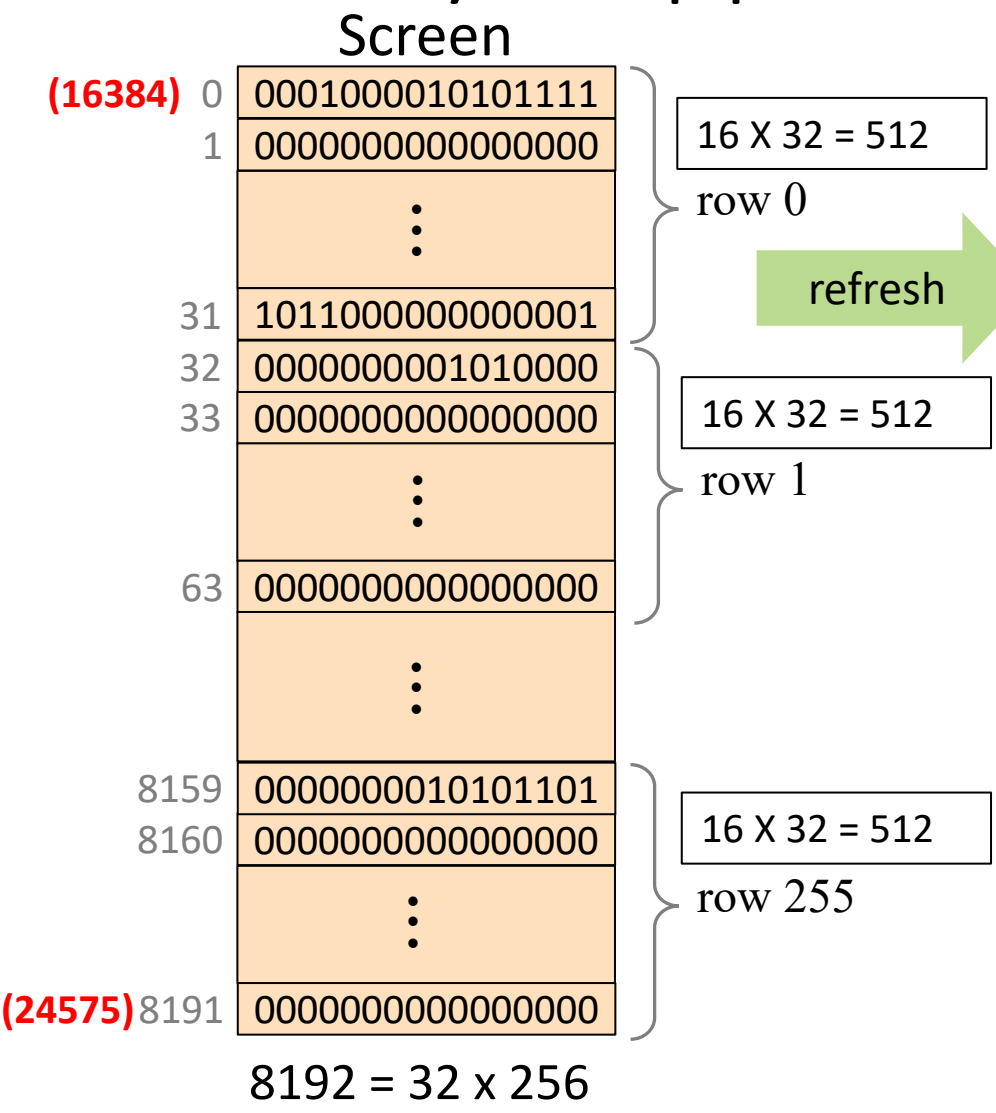
Memory mapped output

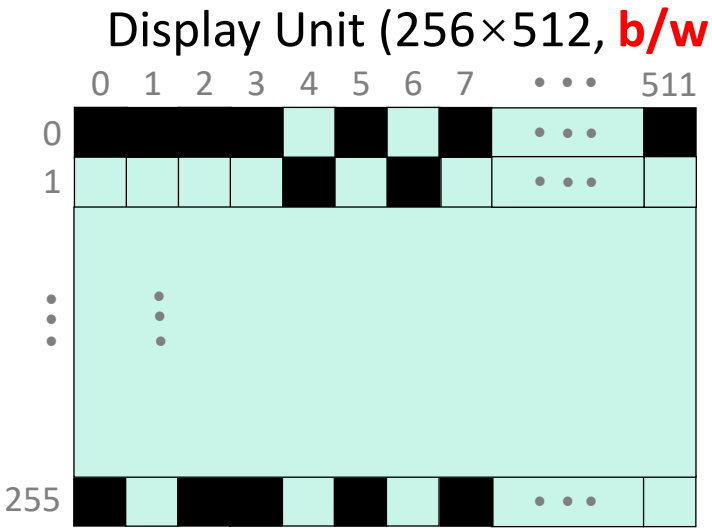


Hack language convention:

- SCREEN: base address of the screen memory map

Memory mapped output



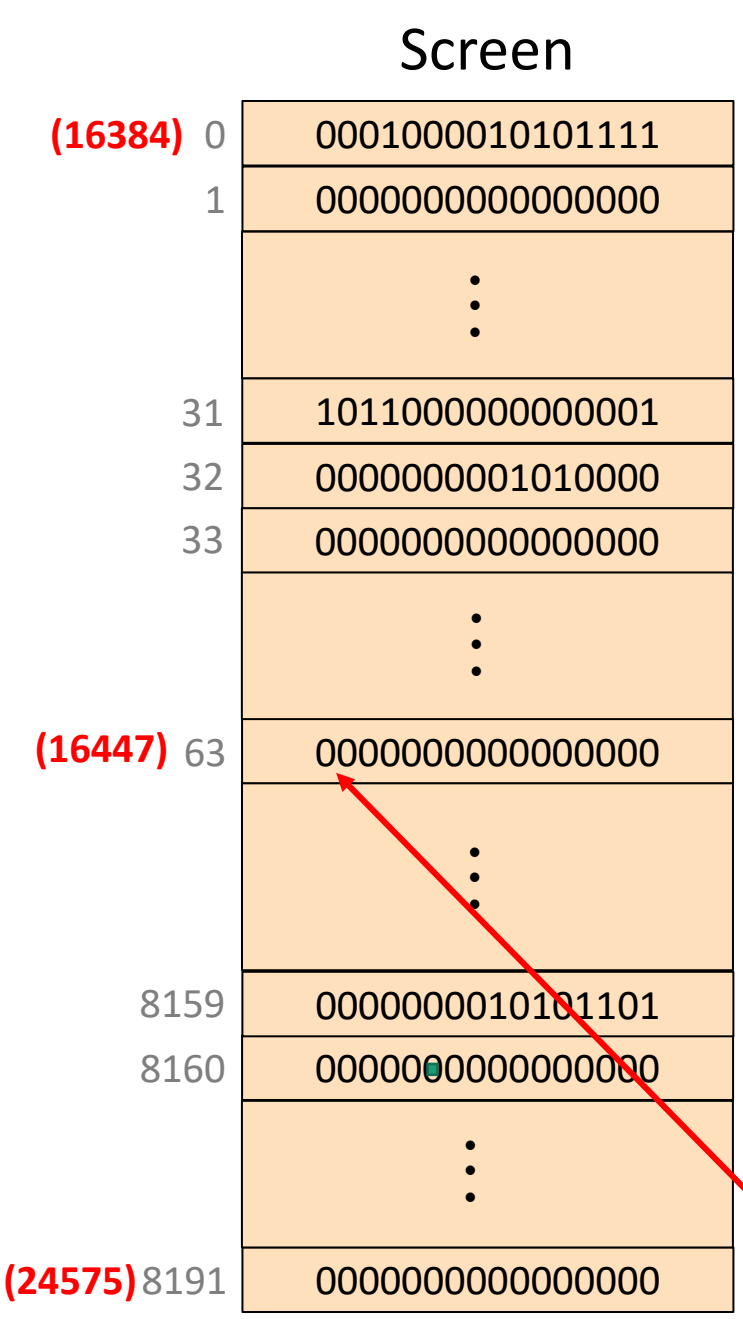


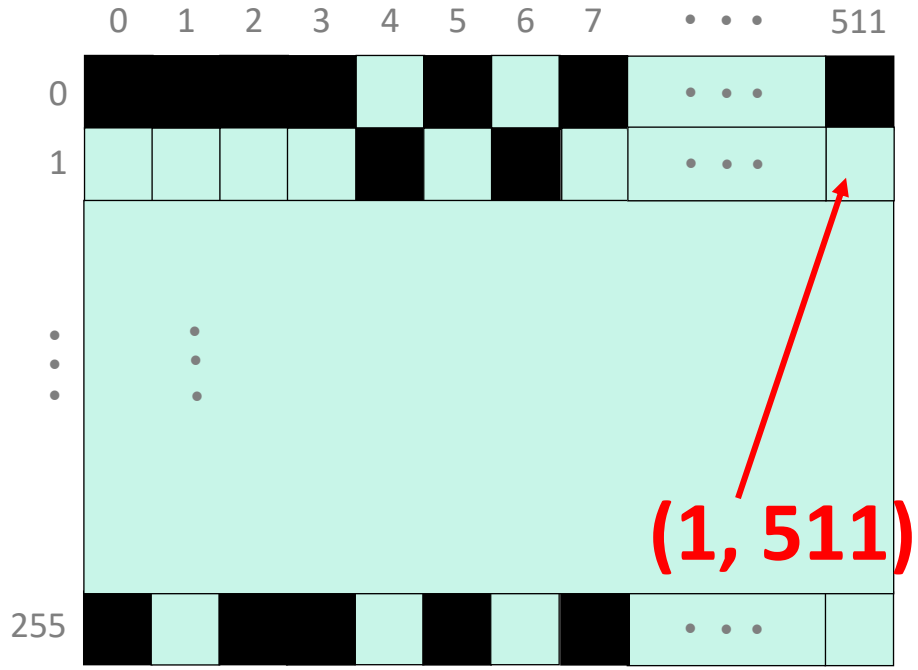
Integer division, round-down.

To set pixel (*row*,*col*) on/off:

(1) *word* = RAM[16384 + 32**row* + *col*/16]

(2) Set the (*col* % 16)*th* bit of *word* to 0 (white) or 1 (black).





(1) *word* = RAM[16384 + 32**row* + *col*/16]

1 511

word=63 32 31

(2) Set the (*col* % 16)*th* bit of *word* to 0 (white) or 1 (black).

511

bit=15

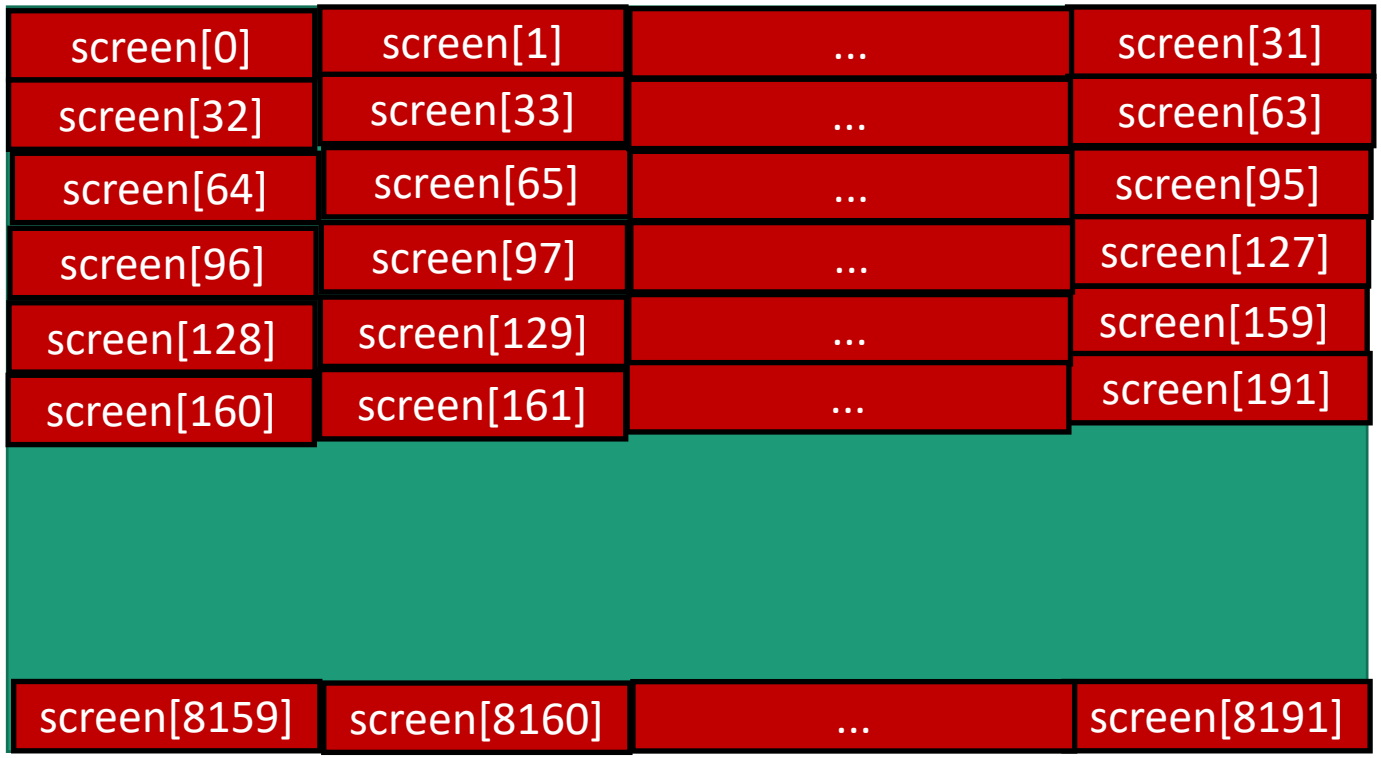
Hack Screen

512-bit wide

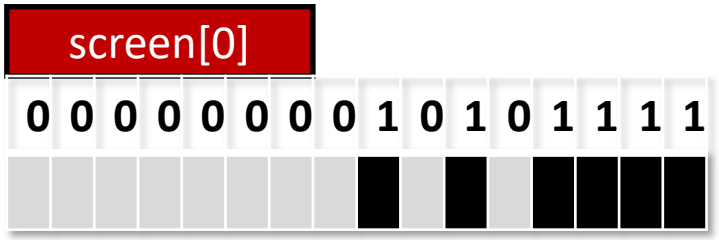
col→

256-bit high

row↓



screen[0] = 1111010100000000

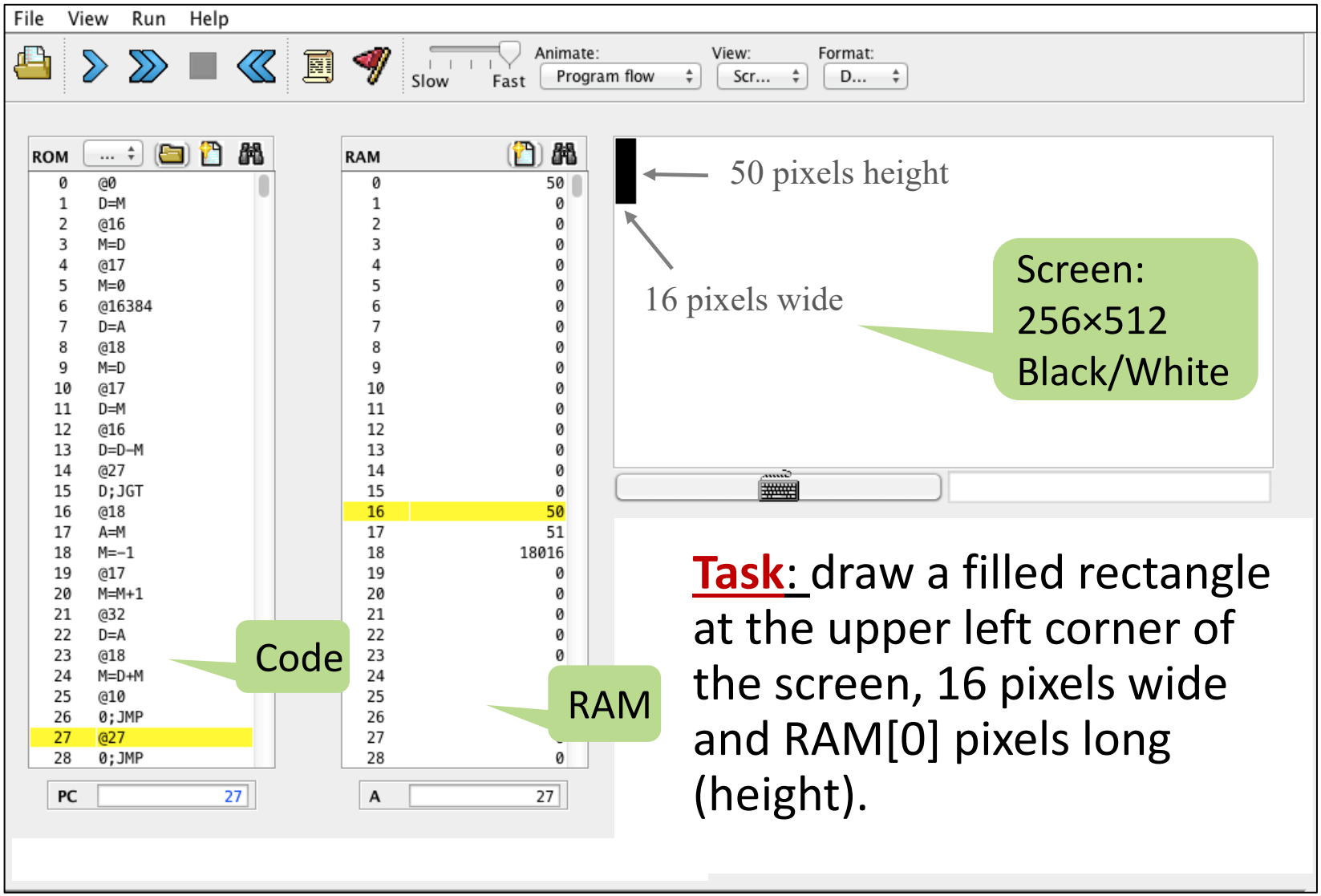


1 for black

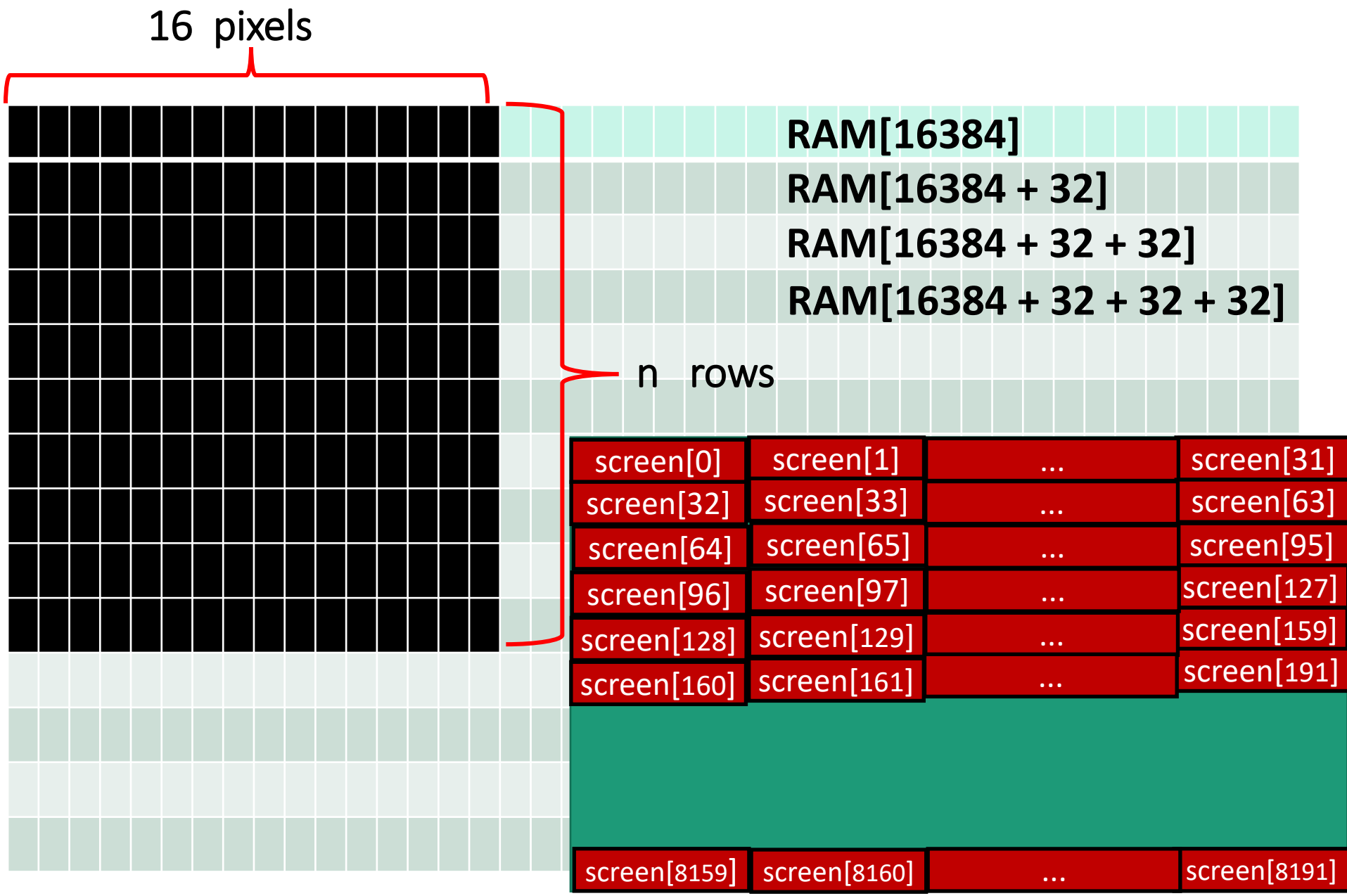
0 for white

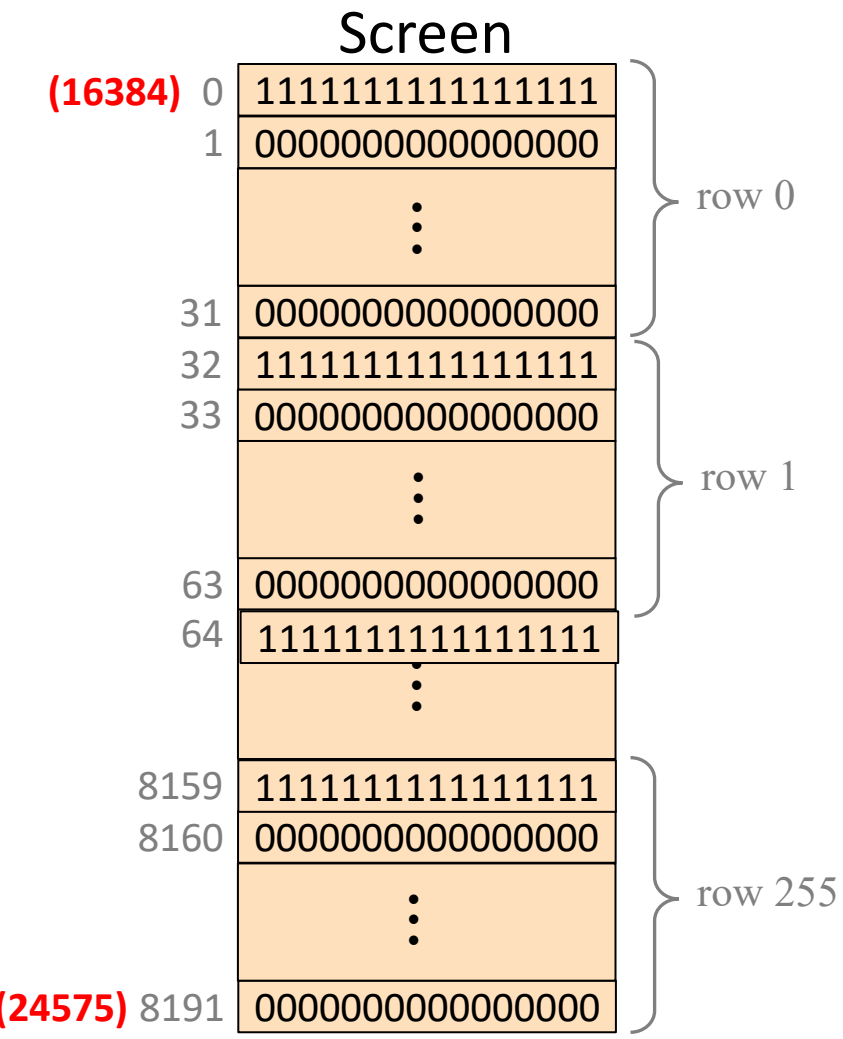
Bit[0], Bit[1], ..., Bit[15]

Handling the screen (example)



Task: draw a filled rectangle at the upper left corner of the screen, 16 pixels wide and RAM[0] pixels long (height).


16 pixels
RAM[16384]
RAM[16384 + 32]
RAM[16384 + 32 + 32]
RAM[16384 + 32 + 32 + 32]
n rows
screen[0]
screen[1]
...
screen[31]
screen[32]
screen[33]
...
screen[63]
screen[64]
screen[65]
...
screen[95]
screen[96]
screen[97]
...
screen[127]
screen[128]
screen[129]
...
screen[159]
screen[160]
screen[161]
...
screen[191]
screen[8159]
screen[8160]
...
screen[8191]



Two's Complement

1 = 0000000000000001

-1 = 1111111111111111

Handling the screen (example)

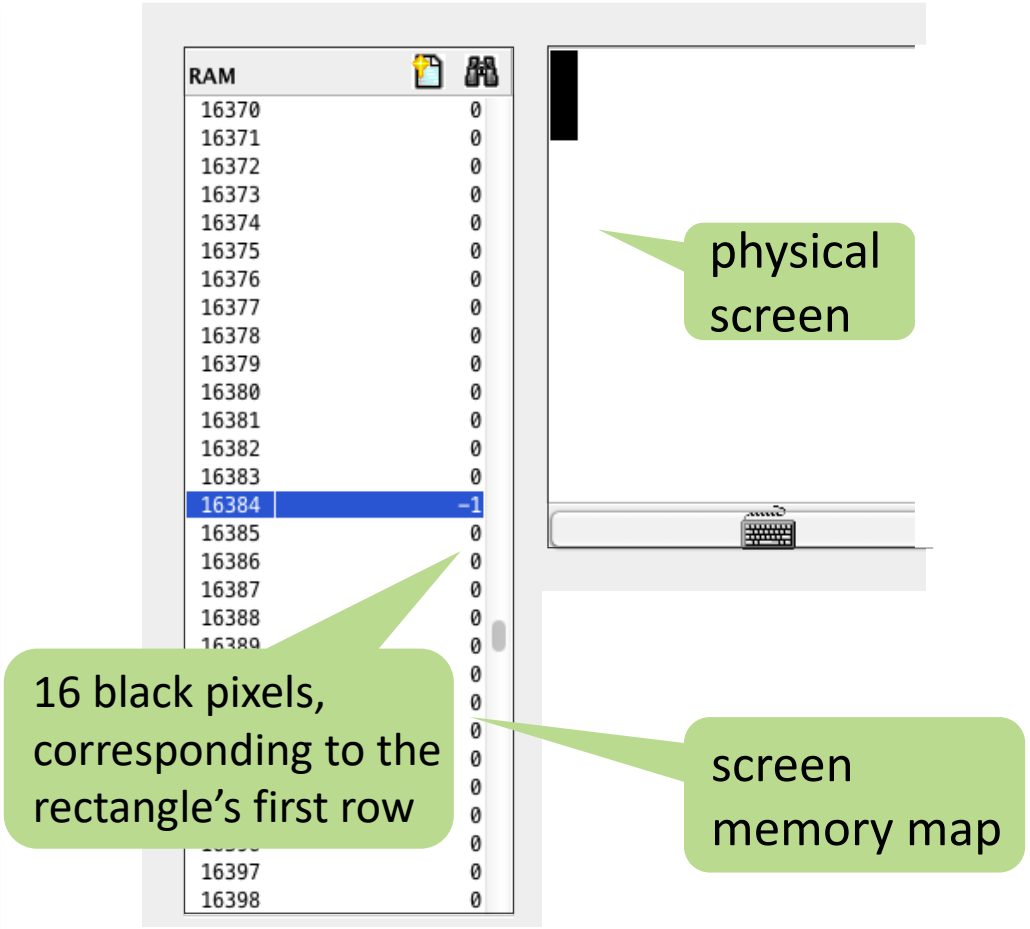
Pseudo code

```
// for (i=0; i<n; i++) {
//     draw 16 black pixels at the
//     beginning of row i
// }

addr = SCREEN
n = RAM[0]
i = 0

LOOP:
   if i == n goto END
   RAM[addr] = -1  //
      1111111111111111
   // advances to the next row
   // 512 = 16 × 32
   addr = addr + 32
   i = i + 1
   goto LOOP

END:
   goto END
```



Handling the screen (example)

Assembly code

```
// Program: Rectangle.asm
// Draws a filled rectangle at the
// screen's top left corner, with
// width of 16 pixels and height of
// RAM[0] pixels.
// Usage: put a non-negative number
// (rectangle's height) in RAM[0].

   @SCREEN
   D=A
   @addr
   M=D  // addr = 16384
      // (screen's base address)
   @R0
   D=M
   @n
   M=D  // n = RAM[0]

   @i
   M=0  // i = 0
```

```
(LOOP)
   @i
   D=M
   @n
   D=D-M
   @END
   D;JEQ  // if i==n goto END

   @addr
   A=M
   M=-1   // RAM[addr]=1111111111111111

   @i
   M=M+1  // i = i + 1
   @32
   D=A   // D = 32
   @addr
   M=D+M  // addr = addr + 32
   @LOOP
   0;JMP  // goto LOOP

(END)
   @END   // program's end
   0;JMP  // infinite loop
```

Handling the screen (example)

Pseudo code

```
// for (i=0; i<n; i++) {
//     draw 16 black pixels at the
//     beginning of row i
// }

addr = SCREEN
n = RAM[0]
i = 0

LOOP:
  if i == n goto END
  RAM[addr] = -1  //
    1111111111111111
  // advances to the next row
  // 512 = 16 × 32
  addr = addr + 32
  i = i + 1
  goto LOOP

END:
  goto END
```

Assembly code

```
// Program: Rectangle.asm
// Draws a filled rectangle at the
// screen's top left corner, with
// width of 16 pixels and height of
// RAM[0] pixels.
// Usage: put a non-negative number
// (rectangle's height) in RAM[0].

  @SCREEN
  D=A
  @addr
  M=D  // addr = 16384
     // (screen's base address)
  @R0
  D=M
  @n
  M=D  // n = RAM[0]

  @i
  M=0  // i = 0
```

Handling the screen (example)

Pseudo code

```
// for (i=0; i<n; i++) {
//     draw 16 black pixels at the
//     beginning of row i
// }

addr = SCREEN
n = RAM[0]
i = 0

LOOP:
  if i == n goto END
  RAM[addr] = -1  //
     1111111111111111
  // advances to the next row
  // 512 = 16 × 32
  i = i + 1
  addr = addr + 32
goto LOOP

END:
  goto END
```

Assembly code

```
(LOOP)
  @i
  D=M
  @n
  D=D-M
  @END
  D;JEQ  // if i==n goto END

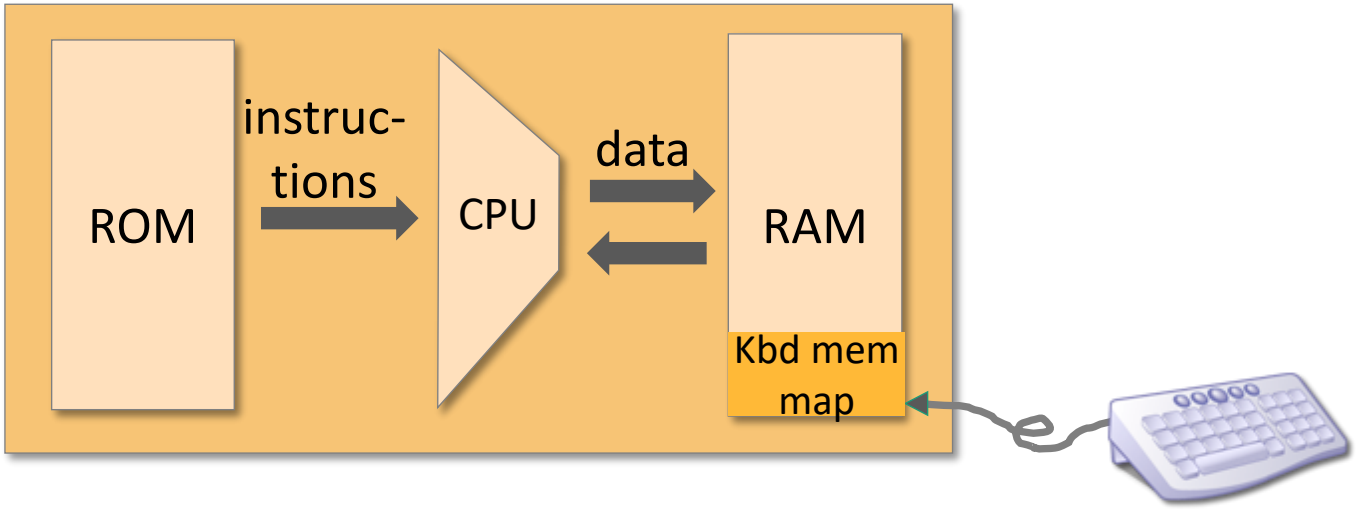
  @addr
  A=M
  M=-1   // RAM[addr]=1111111111111111

  @i
  M=M+1  // i = i + 1

  @32
  D=A   // D = 32
  @addr
  M=D+M  // addr = addr + 32
  @LOOP
  0;JMP  // goto LOOP

(END)
  @END   // program's end
  0;JMP  // infinite loop
```

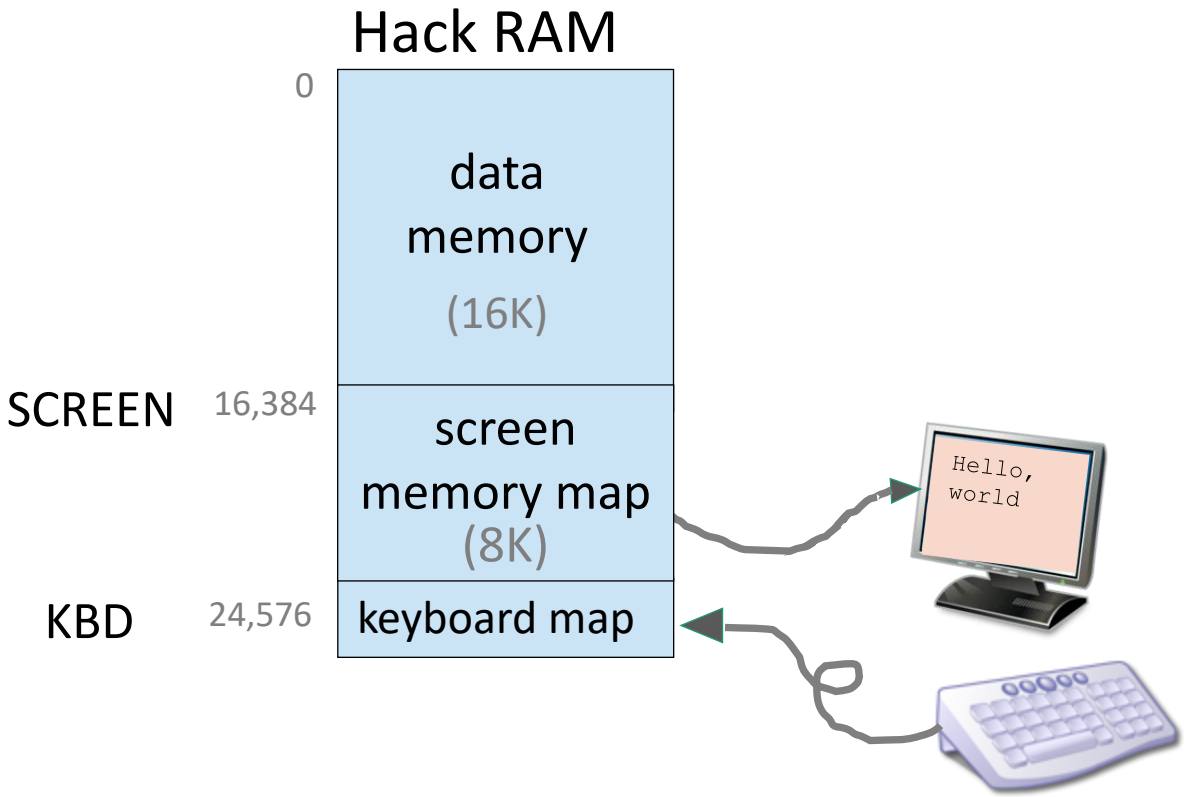
Memory mapped input



Keyboard: used
to enter inputs

The physical keyboard is associated with a *keyboard memory map*.

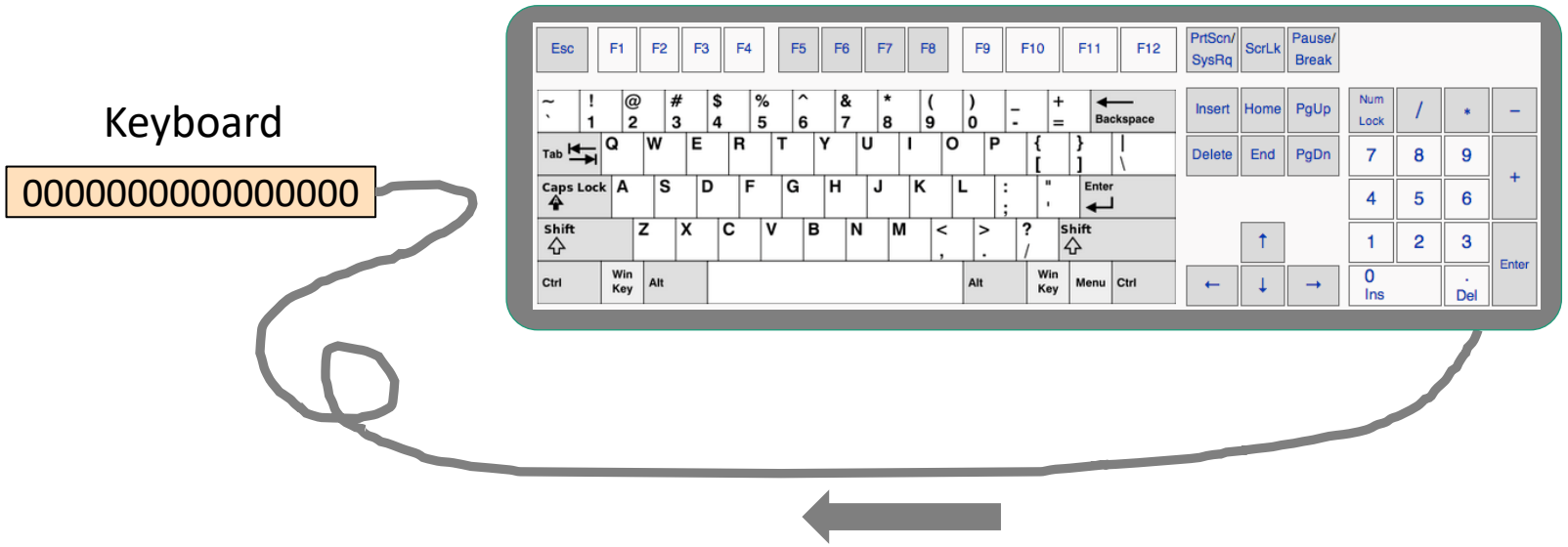
Memory mapped input



Hack language convention:

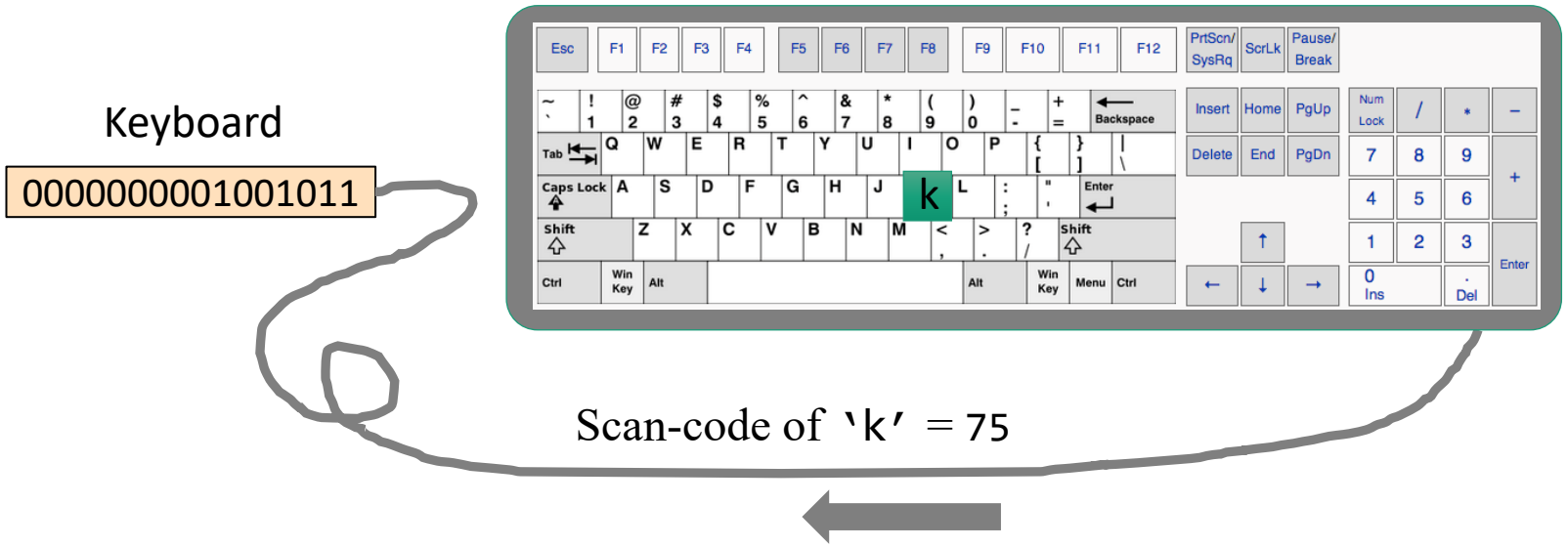
- SCREEN: base address of the screen memory map
- KBD: address of the keyboard memory map

Memory mapped input



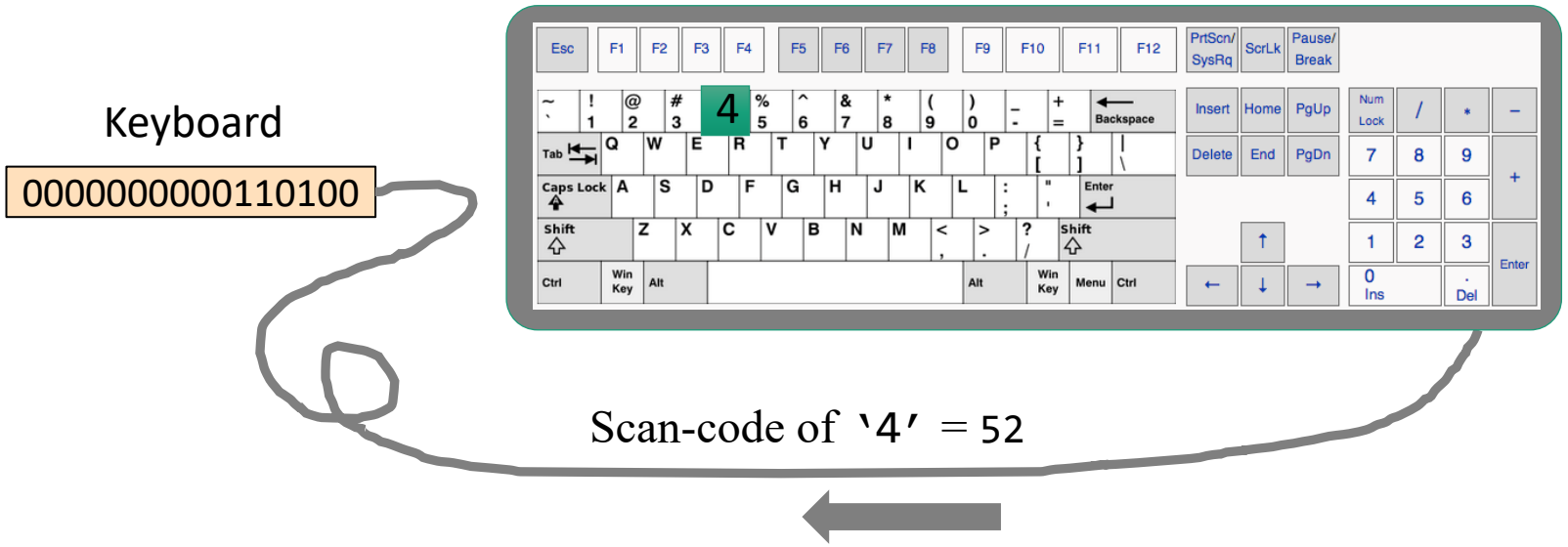
- When a key is pressed on the keyboard, the key's *scan code* appears in the *keyboard memory map.*

Memory mapped input



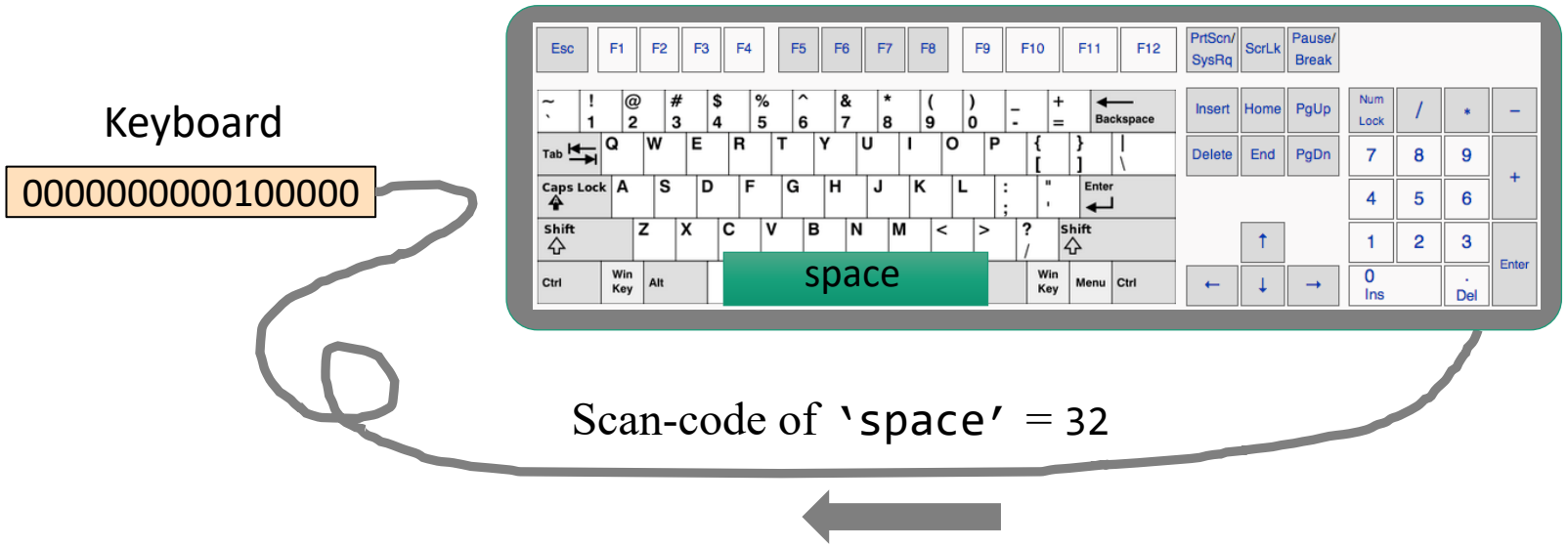
- When a key is pressed on the keyboard, the key's *scan code* appears in the *keyboard memory map.*

Memory mapped input



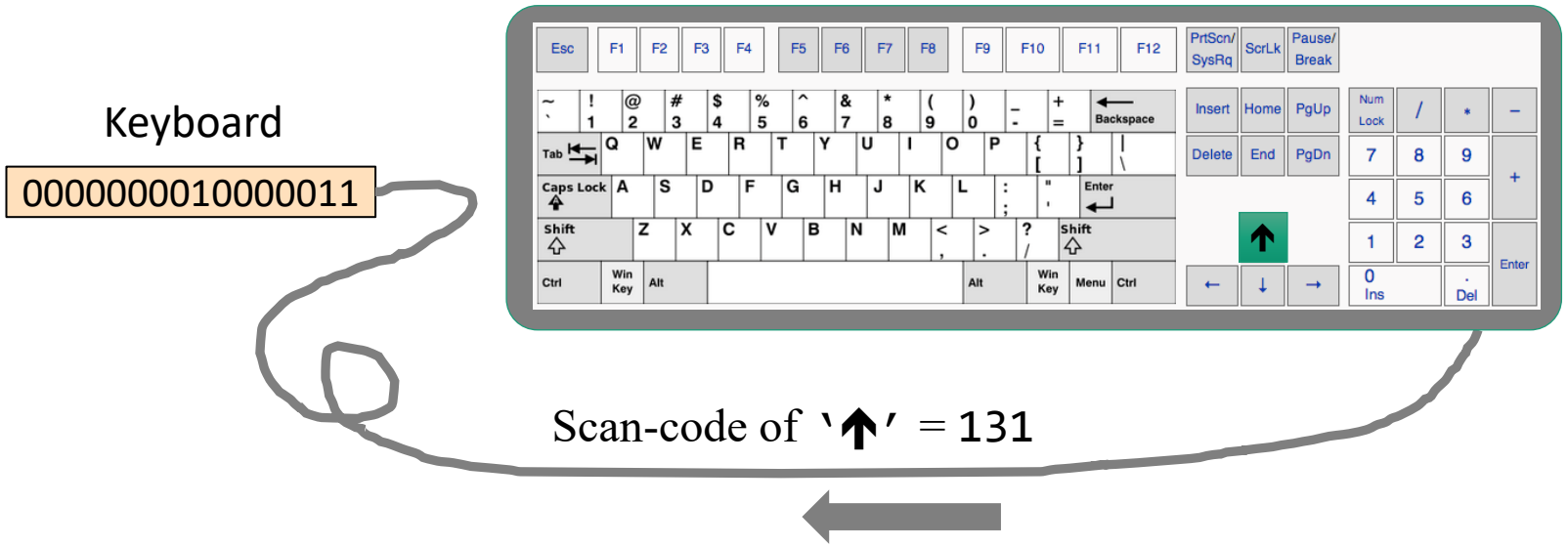
- When a key is pressed on the keyboard, the key's *scan code* appears in the *keyboard memory map.*

Memory mapped input



- When a key is pressed on the keyboard, the key's *scan code* appears in the *keyboard memory map.*

Memory mapped input



- When a key is pressed on the keyboard, the key's *scan code* appears in the *keyboard memory map.*
- When no key is pressed, the resulting code is 0.

The Hack character set

key	code
(space)	32
!	33
“	34
#	35
$	36
%	37
&	38
‘	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47

key	code
0	48
1	49
…	…
9	57

:	58
;	59
<	60
=	61
>	62
?	63
@	64

key	code
A	65
B	66
C	…
…	…
Z	90

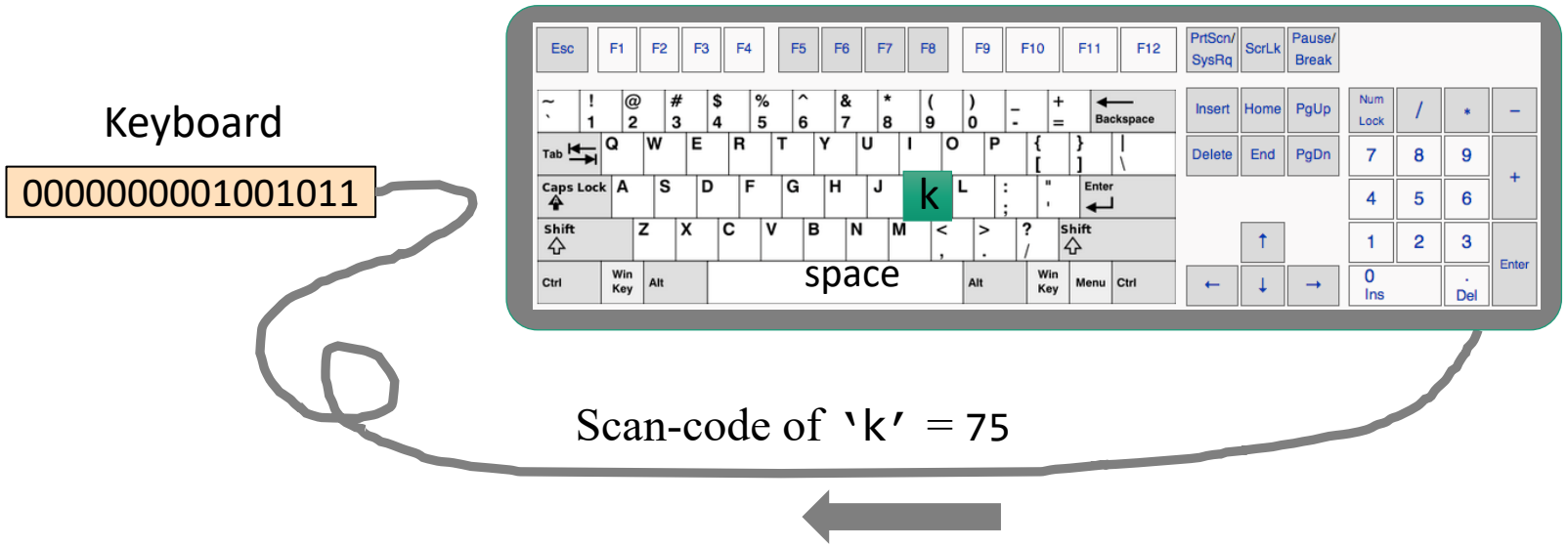
[	91
/	92
]	93
^	94
_	95
`	96

key	code
a	97
b	98
c	99
…	…
z	122

{	123
|	124
}	125
~	126

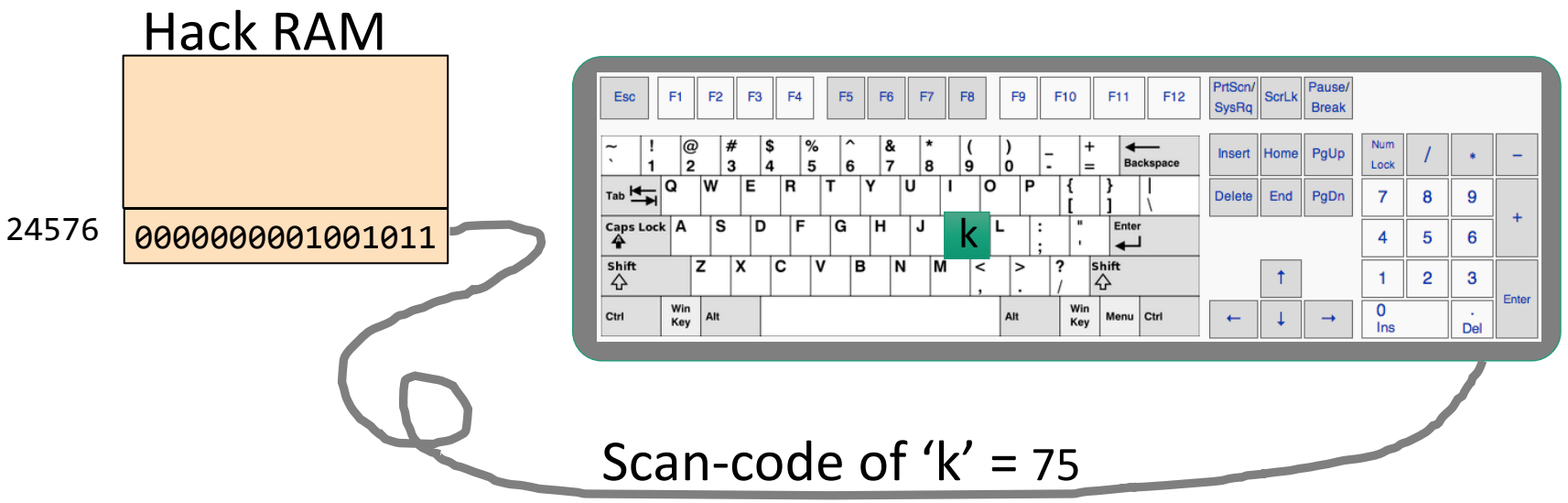
key	code
newline	128
backspace	129
left arrow	130
up arrow	131
right arrow	132
down arrow	133
home	134
end	135
Page up	136
Page down	137
insert	138
delete	139
esc	140
f1	141
…	…
f12	152

Handle the keyboard



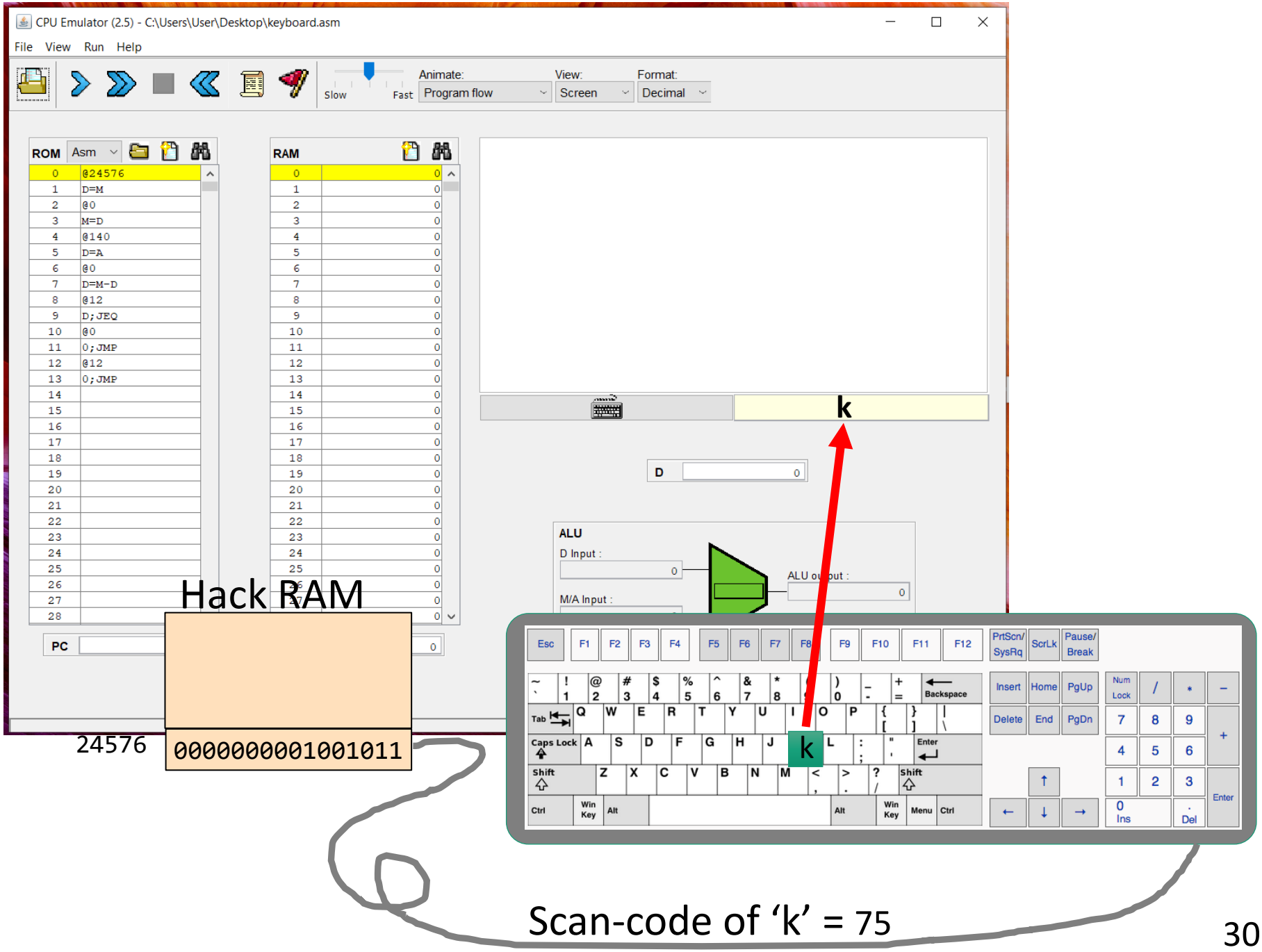
- To check which key is currently pressed:
  - In the Hack computer: probe the contents of **RAM[24576]**.

Handle the keyboard



To check which key is currently pressed:

- Read the contents of `RAM[24576]` (address `KBD`).
- If the register contains `0`, no key is pressed.
- Otherwise, the register contains the scan code of the currently pressed key.


CPU Emulator (2.5) - C:\Users\User\Desktop\keyboard.asm
File View Run Help
Animate: Program flow
View: Screen
Format: Decimal
Slow Fast
ROM Asm
RAM
k
D
ALU
D Input :
M/A Input :
ALU output :
PC
Hack RAM
24576
0000000001001011
k
Scan-code of ‘k’ = 75

Keyboard input (example)

```
// Example: Run an infinite loop to listen to the
      keyboard input
(LOOP)
  // check keyboard input
  @KBD
  D = M //get keyboard input

  @R0
  M=D  //set R0 to keyboard input

  //if R0 = 'esc', goto END
  @140 // 'esc' = 140
  D=A
  @R0
  D=M-D
  @END
  D;JEQ

  @LOOP
  0;JMP // an infinite loop.
(END)
  @END
  0;JMP //end
```

Acknowlegement

- This set of lecture notes are based on the lecture notes provided by Noam Nisam / Shimon Schocken.
- You may find more information on: www.nand2tetris.org.